

SupplyPrescript

An AI-Powered Supply Chain Decision Support System
Predicting Shipment Delays & Actionable Diagnostics

Built with Python, XGBoost, FastAPI, and Vanilla HTML/CSS/JavaScript

Class Imbalance Challenges Delivery Over 10,000 Medical Shipments

The Supply Chain Logistics Problem

- Core Goal: Predict whether vital medical shipments will be delivered late (Is Delayed) based on features known at or before scheduling.
- Medications At Risk: The SCMS Delivery History dataset tracks the distribution of essential HIV/ARV medicine shipments to Africa and Asia.
- Impact: Late deliveries delay patient care, while over-imputed expedited shipping inflates costs. Real-time predictions enable proactive logistics re-routing.

Dataset Scope

10,324

Historical shipments containing 33 original features, including shipment mode, vendor, product, costs, weights, and scheduled delivery dates.

Target Class Distribution (Severe Imbalance)

11.5% Late vs. 88.5% On-Time

- Delayed Cases: 1,186 shipments | On-Time Cases: 9,138 shipments
- Key Challenge: Extreme class imbalance makes standard accuracy metrics highly deceptive, requiring F1-score optimization.

11-Stage Pipeline Transitions Raw Data into a Tuned Model

01

Ingest & Clean

Days 1-2

Loaded 10,324 rows. Found major missingness in Shipment Mode (360) and Dosage (1,736). Converted text-based weight & freight costs to numeric via NaN coercion.



02

Explore & Eng.

Days 3-6

Conducted EDA on outliers & distributions. Engineered binary target 'Is Delayed' (1,186 late, 9,138 on-time). Extracted scheduled month, year, & weekday features.



03

Preprocess & Split

Days 7-8

Simplified 7 high-cardinality columns (Country, Vendor, etc.) to Top 15 + 'Other'. Applied one-hot encoding. Stratified 80/20 train/test split (X_train: 8259, X_test: 2065).



04

Model & Tune

Days 9-11

Established Logistic Regression baseline. Trained XGBoost classifier. Optimized hyper-parameters via 5-fold CV. RandomizedSearchCV focusing on F1-score.



05

Evaluate & Save

Days 12-14

Analyzed ROC-AUC trade-offs and feature importances. Tuned model achieved 0.43 F1. Saved final model as xgboost_best_model.pkl for deployment.

Tuned XGBoost Achieves 0.43 F1 Score, Responding to Class Imbalance

Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC
Baseline (LogReg)	0.885	0.000	0.000	0.000	0.658
XGBoost (Untuned)	0.895	0.579	0.308	0.402	0.903
XGBoost (Tuned)	0.896	0.579	0.342	0.430	0.882

Note: Bold values denote the best model metric in that evaluation category. ROC-AUC (Receiver Operating Characteristic Area Under Curve) assesses model ranking performance, while F1-score balances precision and recall on the minor 'delayed' class.

Key Evaluation Insights

- The Accuracy Trap: The baseline Logistic Regression model achieves 88.52% accuracy simply by predicting 'On Time' for every shipment, failing to capture a single delayed shipment (0.0 F1).
- XGBoost Advantage: XGBoost substantially resolves class imbalance, improving the F1 score to 0.402 (untuned) and 0.430 (tuned).
- F1 vs. ROC-AUC Trade-off: Tuning optimized F1 (threshold-dependent) but slightly decreased ranking quality (ROC-AUC from 0.903 to 0.882), showing optimization metrics trade off.
- Model Features: Feature importances reveal that shipment origin/node (Fulfill Via_From RDC) and destination (South Africa, Nigeria) are the primary predictors of delivery delay.

Three-Tier Architecture Enables Real-Time Custom Predictions

Frontend Interface

Stack: HTML5, CSS3, JavaScript

- Interactive Dashboard: Allows users to input Country, Shipment Mode, Weight, and Freight Cost.
- Client-side Validation: Restricts forms from blank submissions and negative numeric inputs.
- Graceful Fallbacks: Automatically detects when the API is down and displays a clear, styled 'Backend not running' message rather than crashing.

Backend API Core

Stack: FastAPI (Python)

- RESTful Endpoints: Exposes GET / (health), POST /predict (raw), POST /predict_custom (frontend-facing), and GET /test (demo prediction).
- Translation Layer: Maps user inputs and implements build_feature_row() to handle missing features.
- CORS Security: Hardcoded to dashboard origin (<http://127.0.0.1:5500>) to block cross-origin requests.

Machine Learning Model

Stack: XGBoost Classifier (.pkl)

- Model Deserialization: Backend loads xgboost_best_model.pkl once at startup using joblib.
- Prediction Execution: Scores 157-element preprocessed rows representing shipment configurations.
- Stable Performance: Relies on RDC fulfillment, vendor terms, and destination country encodes for prediction.

Four Core Features Override Training Defaults to Yield Predictions

The Translation Layer Mechanism

Because the trained model has 157 feature dimensions after one-hot encoding, forcing a user to enter 157 variables is impossible. The api.py solves this via a robust default mapping layer:

- `_compute_training_defaults()`: At server startup, the backend computes the median values for numerical columns and the most frequent categories for categorical columns using `x_train.csv`.
- Custom Inputs: The user only inputs 4 critical features: Country, Shipment Mode, Weight, and Freight Cost.
- `build_feature_row()`: The API constructs a 157-column template representing a 'typical shipment' from training data defaults, then overrides it with the 4 user values before inference.
- Graceful Fallback: If a user enters an unknown country/mode, the API silently falls back to training defaults instead of crashing.

From Prediction to Business Action

When the user clicks 'Predict This Shipment', the frontend parses the model's binary response (0 = On Time, 1 = Delay Expected) and triggers active decision cards:

✓ Air Freight (Expedited)

Cost: $1.5 \times \text{Custom Freight Cost}$ | Speed: Fast
Automatically scales based on the user's freight cost input to simulate real rush transport pricing.

✓ Secondary Supplier

Cost: +10% | Speed: Medium
Calculates a 10% premium on the current shipment cost to trigger a localized back-up supplier order.

✓ Delay Launch (Buffer)

Cost: \$0 | Speed: Slow
Triggers a cost-free scheduling buffer, notifying downstream clinical receivers of a minor delay.

CORS Hardcoding & 0.34 Recall Highlight

Essential Production Gaps

Current Limitations

- **Model Sensitivity (Recall 0.34):** The best model misses roughly two-thirds of actual delayed shipments. This reflects the inherent difficulty of predicting rare supply chain events (~11.5% delay rate) with limited scheduling features.
- **Translation Feature Bias:** The `/predict_custom` endpoint only exposes 4 features. The remaining 153 features use general training averages, which might mask specific shipment contexts (such as specific vendor delays).
- **API Security & CORS:** The backend CORS origin is hardcoded to `http://127.0.0.1:5500` in `api.py`, which blocks external clients. No authentication is implemented.
- **Missing Backend Re-validation:** While JavaScript checks for positive weight and costs, the backend accepts raw negative numbers if the API is called directly.

Future Technical Roadmap

1. **Active Database Integration (PostgreSQL)**
Integrate the planned database layer to log prediction queries, track actual vs. predicted shipment outcomes, and capture new samples for incremental training.
2. **Comprehensive Input Feature Exposure**
Expand the user interface and `/predict_custom` endpoint to collect vendor name, product group, and therapeutic dosage, reducing reliance on training-data averages.
3. **Hardened API Security & Input Validation**
Implement JWT authentication to restrict endpoint access, dynamically configure CORS variables, and move validation logic to the server side (Pydantic / FastAPI).
4. **Advanced Operations Research Solver**
Replace the current direct multipliers for recommended actions (e.g. 1.5× cost) with a programmatic scheduling and routing optimization solver (PuLP / SciPy).